



Microprocesadores y Microcontroladores: Proyecto Final

El Algoritmo Goertzel – una alternativa de la FFT

(14 de junio del 2026)

Yahir Alejandro Baeza Collazo

ya.baezacollazo@ugto.mx

Resumen -En el siguiente proyecto se desarrolla la implementación del algoritmo Goertzel por medio de la tarjeta NUCLEO-L476RG para detectar dos frecuencias diferentes de entrada.

suma de senos y cosenos de diferentes frecuencias y amplitudes desfasadas en el tiempo. La Ecuación 1 muestra la definición de la transformada discreta de Fourier [1].

I. ANTECEDENTES

El algoritmo de Goertzel es una técnica de procesamiento digital de señales para la evaluación eficiente de los términos individuales de la transformada discreta de Fourier. El algoritmo fue descrito por primera vez por Gerald Goertzel en 1958, al igual que la transformada discreta de Fourier, el algoritmo de Goertzel analiza un componente de frecuencia seleccionable de una señal discreta, la diferencia radica en que el algoritmo de Goertzel aplica un único coeficiente de valor real en cada iteración, utilizando aritmética de valor real para secuencias de valores reales de entrada [2].

El algoritmo de Goertzel tiene un orden de complejidad mayor que los algoritmos de transformada rápida de Fourier, sin embargo, para calcular un pequeño número de componentes de frecuencia seleccionados, es numéricamente más eficiente. El algoritmo de Goertzel fue propuesto con el propósito de realizar la evaluación de series trigonométricas finitas, con el paso del tiempo este algoritmo se implementó como un filtro para detectar la presencia de señal en un ancho de banda muy reducido, independientemente del resto del espectro que lo rodea. Implementar filtros pasabandas de 100 a 200 Hz de ancho, pero con la particularidad de requerir muchísimas menos operaciones matemáticas que las necesarias para un filtro equivalente utilizando algoritmos más convencionales [3].

II. ALGORITMO GOERTZEL

El algoritmo de Goertzel parte de la transformada de Fourier, se sabe que esta transformada descompone la señal como la

$$x(k) = \sum_{n=0}^{N-1} x(n) e^{\left(\frac{-2\pi kn}{N}\right)j} \quad (1)$$

N = Número total de muestras

n = *enésima muestra original*

k = *késimo término de la transformada discreta de fourier*

El algoritmo de Goertzel parte de la definición de la transformada discreta de Fourier, por lo tanto, se procede a realizar la rescritura de la Ecuación 1. Si se define a [1]:

$$W_N^k = e^{\left(\frac{-2\pi k}{N}\right)j}$$

$$W_N^{kn} = \left[e^{\left(\frac{-2\pi k}{N}\right)j} \right]^n$$

$$W_N^{kn} = e^{\left(\frac{-2\pi kn}{N}\right)j}$$

Por lo que la Ecuación 1 se reescribe de la siguiente manera:

$$x(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad (2)$$

Como:

$$W_N^{-kn} = [W_N^k]^{-N} = \left[e^{\left(\frac{-2\pi k}{N}\right)j} \right]^{-N} = e^{\left(\frac{-2\pi Nk}{N}\right)j} = e^{(-2\pi k)j}$$

Y desarrollando mediante la identidad de Euler:

$$e^{(-2\pi k)j} = \cos(-2\pi k) + \sin(-2\pi k)j = 1$$

Por lo tanto, se obtiene que:

$$W_N^{-kN} = 1 \quad (3)$$

Haciendo uso de la identidad de Euler, representada en la Ecuación 3, sobre la Ecuación 2, se obtiene:

$$x(k) = \sum_{n=0}^{N-1} x(n)[1]W_N^{kn}$$

$$x(k) = \sum_{n=0}^{N-1} x(n)W_N^{-kN}W_N^{kn}$$

$$x(k) = \sum_{n=0}^{N-1} x(n)W_N^{-kN+kn}$$

Factorizando se obtiene la Ecuación 4:

$$x(k) = \sum_{n=0}^{N-1} x(n)W_N^{-k(N+n)} \quad (4)$$

Desarrollando la sumatoria resultante y factorizando se obtiene la Ecuación 5:

$$x(k) = \{[x(0)W_N^{-k} + x(1)]W_N^{-k} + x(2)W_N^{-k} + \dots + x(N-1)W_N^{-k}\} \quad (5)$$

Expresando como una ecuación de diferencias la Ecuación 5, se obtiene la Ecuación 6:

$$y(n) = W_N^{-k}y(n-1) + x(n) \quad (6)$$

Donde $y(n)$ representa la salida y $y(n-1)$ la salida anterior y $x(n)$ la entrada. Aplicando la transformada Z a la Ecuación 6 de diferencias, se obtiene:

$$y(z) = W_N^{-k}y(z)Z^{-1} + x(z)$$

$$y(z) - W_N^{-k}y(z)Z^{-1} = x(z)$$

$$y(z)[1 - W_N^{-k}Z^{-1}] = x(z)$$

Donde la Ecuación 7 es la función de transferencia resultante:

$$H(z) = \frac{y(z)}{x(z)} = \frac{1}{1 - W_N^{-k}Z^{-1}} \quad (7)$$

Multiplicado el numerador y el denominador de la función de transferencia de la Ecuación 7 por:

$$1 - W_N^k Z^{-1}$$

Se obtiene que:

$$H(z) = \frac{y(z)}{x(z)} = \frac{1 - W_N^k Z^{-1}}{(1 - W_N^{-k} Z^{-1})(1 - W_N^k Z^{-1})}$$

$$H(z) = \frac{1 - W_N^k Z^{-1}}{1 - [(W_N^{-k} + W_N^k)Z^{-1} - Z^{-2}]}$$

$$H(z) = \frac{1 - W_N^k Z^{-1}}{1 - [2 \left(\cos \left(\frac{2\pi k}{N} \right) \right) Z^{-1} - Z^{-2}]}$$

Así se llega a la Ecuación 8 que corresponde al filtro derivado del algoritmo Goertzel:

$$H(z) = \frac{1 - W_N^k Z^{-1}}{1 - 2 \left(\cos \left(\frac{2\pi k}{N} \right) \right) Z^{-1} - Z^{-2}} \quad (8)$$

En la Figura 1 se muestra el diagrama del filtro IIR del algoritmo Goertzel de la Ecuación 8.

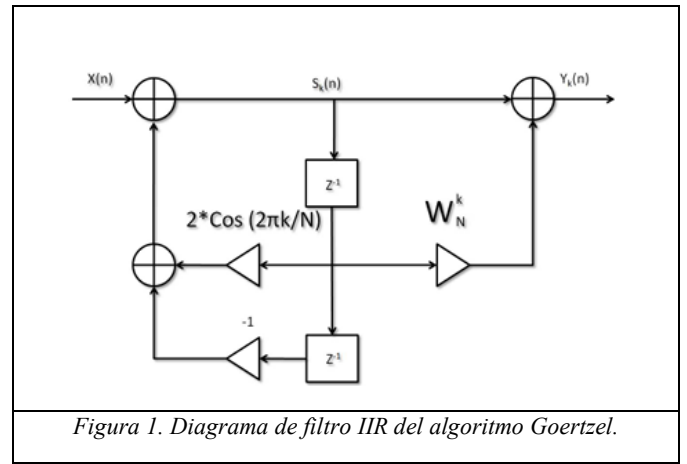


Figura 1. Diagrama de filtro IIR del algoritmo Goertzel.

Expresando la Ecuación 8 como diferencias, se obtiene la Ecuación 9, que resulta ser una expresión en diferencias del algoritmo:

$$\{1 - 2 \left(\cos \left(\frac{2\pi k}{N} \right) \right) Z^{-1} - Z^{-2}\} H(z) = 1 - W_N^k Z^{-1} \quad (9)$$

$$y(n) - 2 \cos \left(\frac{2\pi k}{N} \right) y(n-1) + y(n-2) = x(n) - W_N^k x(n-1)$$

Con el objetivo de implementar el algoritmo, la Ecuación 9, se descompone como las Ecuaciones 10 y 11.

$$S(n) = x(n) + 2 \cos \left(\frac{2\pi k}{N} \right) y(n-1) + y(n-2) \quad (10)$$

$$Y(n) = S(n) - W_N^k x(n-1) \quad (11)$$

Para implementar las Ecuaciones 10 y 11 en el programa, se deben de escribir de la siguiente manera:

$$S_k(n) = x(n) + 2 \cos \left(\frac{2\pi k}{N} \right) S_k(N-1) + S_k(N-2) \quad (12)$$

$$Y_k(n) = S_k(n) - W_N^k S_k(n-1) \quad (13)$$

La ecuación 14 calcula la transformada discreta de Fourier para el k -ésimo término para la frecuencia del

componente espectral que se desea encontrar con la frecuencia de muestreo de una señal de N muestras:

$$k = N \frac{F_x}{F_s} \quad (14)$$

III. MATERIAL

- Tarjeta de evaluación NUCLEO-L476RTG
- Cable mini-USB (Datos)
- Jumpers
- Computadora
- STM32CubeMX
- STM32CUBEIDE
- Generador de Funciones
- Osciloscopio

IV. DESARROLLO

A. Configuración de operación del microcontrolador.

Para iniciar con el desarrollo Del proyecto, es necesario configurar el hardware con los periféricos por defecto que nos da el STM32CubeMX, estos son el Led verde (**LD2**), el botón azul (**B1**) y el puerto de comunicación asíncrona (**UART2**). Además, se requiere la activación de un canal de un ADC configurado para transferir el conjunto de las conversiones a través del DMA.

a. Selección de tarjeta.

El desarrollo de la práctica comienza en el software STM32CUBEMX, al ejecutarlo se seleccionó la tarjeta **NUCLEO-L476RG** y se aceptó la configuración de periféricos por defecto.

b. Configuración de pines.

La terminal para detectar la señal de entrada fue el puerto **PA0** se configuró como **ADC1_IN5**. Para configurar el ADC1 nos dirigimos a la siguiente ventana **[Analog]→[ADC1]**, después nos dirigimos a las siguientes ventanas:

Mode: [IN5]→[IN5 Single-ended]

[Parameters Settings]→[ADC Regular ConversionMode]

[External Trigger Conversion Source]→[Timer 3 Trigger Out event]

[Rank]→[Sampling Time]→[640.5 Cycles]

La configuración se muestra en la Figura 2.

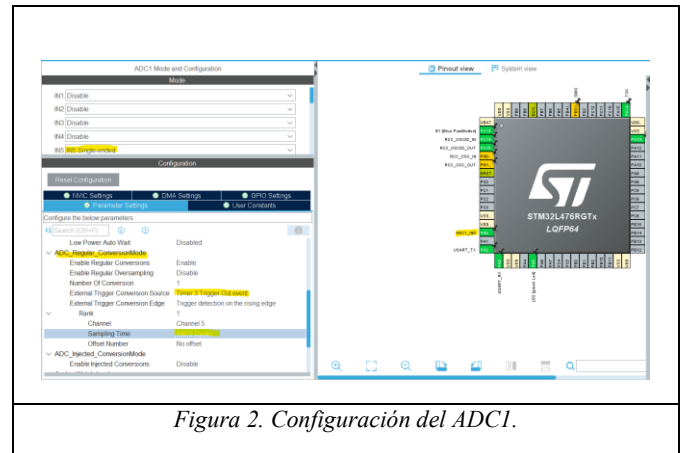


Figura 2. Configuración del ADC1.

Después se configuró el DMA para la transmisión de datos a la memoria por el ADC1, para ello nos dirigimos a la ventana de **[DMA Settings]** y se seleccionará el botón de agregar para activar el canal, entonces se seleccionará la siguiente configuración:

[Select]→[ADC1]

[DMA Request Settings]→[Mode]→[Circular].

[DMARequest Settings]→[DataWidth]→[Peripheral]→[Word]

[DMARequest Settings]→[DataWidth]→[Memory]→[Word]

Se navegará hasta la sección **[NVIC Settings]** y se seleccionará la interrupción **[ADC1 and ADC2interrupts]**.



Figura 3. Configuración del DMA.

Se configurará el temporizador 3 para una frecuencia de 20 KHz, este operará en conjunto con el ADC1. La configuración del temporizador 3 se realiza en el apartado de [Categories] y se selecciona [Timer]→[TIM3], las opciones que se configuraron fueron las siguientes:

[Mode]:

[Clock Source]→[Internal Clock]

[Configuration]→[Parameters Settings]→

[Counter Settings]:

[Prescaler (PSC- 16 bits value)→[79]

[Counter Period (AutoReload Register- 16 bits value)→[49]

[Configuration]→[Parameters Settings]→[Trigger Output (TRGO) Parameters]:

[Trigger Event Selection TRGO]→[Update Event]

c. Configuración de proyecto.

En la ventana de **Project Manager** se le pondrá el nombre que corresponda, se seleccionará la ruta de acceso donde se guardará y finalmente se seleccionará el IDE de **STM32CUBEIDE**, como se observa en la Figura 4.

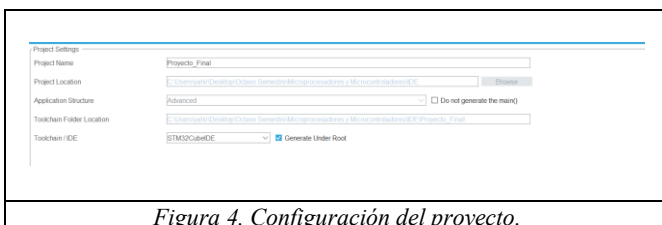


Figura 4. Configuración del proyecto.

En la ventana lateral llamada **Code Generator** se seleccionará la opción de [Generate peripheral initialization as a pair of

‘.c/.h’ files per peripheral], esto permite generar los archivos .c y .h de manera separada.

Finalmente seleccionaremos la ventana de **Generate Code** y aceptaremos las ventanas emergentes que nos aparezcan.

B. Programación de la aplicación en el IDE: **STM32CubeIDE**

a. Incluir librerías.

Las librerías que se utilizaron fueron las siguientes:

```
/* USER CODE BEGIN Includes */
#include "math.h"
/* USER CODE END Includes */
```

b. Definición de constantes.

Se declararon como constantes el número de muestras, la frecuencia de muestreo y las dos frecuencias que se quieren determinar.

```
/* USER CODE BEGIN PD */
#define N 100
#define Fs 20000

#define Fx1 750
#define Fx2 2000
/* USER CODE END PD */
```

c. Declaración de constantes.

Se declaro un arreglo de tamaño 100 que actúa como memoria intermedia donde el DMA deposita de forma directa y automática los datos muestreados por el ADC, como el algoritmo se encarga de procesar dos frecuencias, se utilizan dos vectores de 3 posiciones que guardan los estados del filtro retroalimentado de segundo orden, después se declaran las constantes pre-calculadas en el super ciclo, estas definen la frecuencia exacta que el filtro va a “hacer resonar”. Finalmente se declaran las componentes complejas y de potencia, los cuales son vectores para descomponer la señal final en sus componentes real e imaginaria, permitiendo calcular la magnitud lineal neta detectada.

```
/* USER CODE BEGIN PV */
uint32_t x[N]={};

float s1[3] = {0,0,0};
float cr1, ci1, k1=(float)N*Fx1/Fs;
float Xpower1, y_1[2];
```

```
float s2[3] = {0,0,0};
float cr2, ci2, k2=(float)N*Fx2/Fs;
float Xpower2, y_2[2];

float Xpower_mayor;
/* USER CODE END PV */
```

d. Declaración de función de interrupción.

Se declaro la función de interrupción para declarar los cálculos correspondientes:

```
/* USER CODE BEGIN PFP */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef*hadc);
/* USER CODE END PFP */
```

e. Precálculo de coeficientes matemáticos e inicialización de funciones.

En esta sección se calculan las funciones trigonométricas (coseno y seno) necesarias para sintonizar el filtro de Goertzel en las frecuencias que se buscan determinar. Se modifica el registro ARR del temporizador 3 de forma dinámica, después se calibra el ADC para obtener un menor error en la conversión, se inicializa el DMA, finalmente se inicializa el temporizador 3 como base de tiempo.

```
/* USER CODE BEGIN 2 */
cr1 = cosf(2*M_PI*k1/N);
ci1 = sinf(2*M_PI*k1/N);

cr2 = cosf(2*M_PI*k2/N);
ci2 = sinf(2*M_PI*k2/N);

TIM3->ARR = HAL_RCC_GetPCLK1Freq()/(Fs*(TIM3->PSC +1))-1;

HAL_ADCEX_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)x, N);
HAL_TIM_Base_Start(&htim3);

/* USER CODE END 2 */
```

Dentro de la función de interrupción se verifica primero si la interrupción fue causada por el ADC1, después se detiene el DMA para que no se siga sobre escribiendo el arreglo de las muestras. Se crea un ciclo para calcular cada uno de los elementos para cada frecuencia individual, lo primero que se realiza es transformar los datos digitales a un valor de voltaje entre 0 y 3.3 V, después se procesa la muestra de voltaje a través de una ecuación matemática recursiva de segundo orden para cada frecuencia por separado.

Se calcula la parte real e imaginaria de los estados finales guardados en el bucle junto con los coeficientes trigonométricos para proyectar la señal en un plano complejo. En la variable Xpower aplica el teorema de Pitágoras para obtener la magnitud de la frecuencia analizada; al multiplicar el

resultado por $\frac{2}{N}$, la variable Xpower1 y su contra Xpower2 se convierte directamente en el voltaje pico de esa frecuencia especificada detectada en el aire.

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef*hadc){

    if(hadc->Instance == ADC1){
        HAL_ADC_Stop_DMA(hadc);
        for(uint8_t i=0; i<N; i++){

            //PARA FX1
            s1[2]= 3.3*((float)x[i])/4095 + 2 * cr1 * s1[1] -s1[0];
            s1[0]= s1[1];
            s1[1]= s1[2];

            //PARA FX2
            s2[2]= 3.3*((float)x[i])/4095 + 2 * cr2 * s2[1] -s2[0];
            s2[0]= s2[1];
            s2[1]= s2[2];

        }

        //IMAGINARIO FX1
        y_1[0]= cr1*s1[1]-s1[0];
        y_1[1]= ci1*s1[1];
        Xpower1= powf(y_1[0],2) + powf(y_1[1],2);
        Xpower1= sqrtf(Xpower1);
        Xpower1= 2*Xpower1 / N;

        //IMAGINARIO FX2
        y_2[0]= cr2*s2[1]-s2[0];
        y_2[1]= ci2*s2[1];
        Xpower2= powf(y_2[0],2) + powf(y_2[1],2);
        Xpower2= sqrtf(Xpower2);
        Xpower2= 2*Xpower2 / N;

    }
}
```

f. Calibración de señal con generador de funciones y osciloscopio.

Se configuró una señal de 1.5 V_{pp} y un offset de 1.65 V como se observa en la Figura 5.

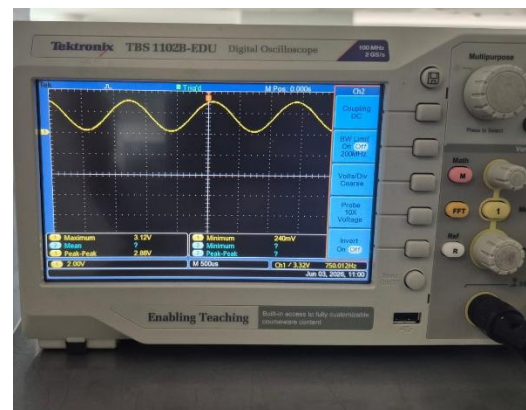


Figura 5. Calibración de señal de entrada.

Las conexiones en la tarjeta se muestran en la Figura 6.

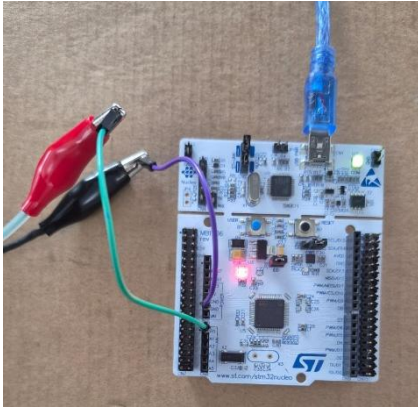


Figura 6. Conexiones físicas.

V. RESULTADOS

Para comprobar que el algoritmo funcionara de manera correcta se configuro una señal de 750 Hz y se verifico por medio del modo depuración el valor de la variable `xpower1`, en este caso el valor estaba alrededor de 1, por lo tanto, el algoritmo había detectado de manera correcta la frecuencia de los 750 Hz. En la Figura 7 se muestra el resultado de la detección de la frecuencia.

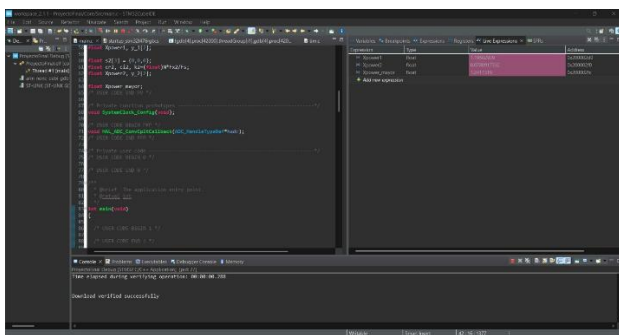


Figura 7. Detección de frecuencia de 750 Hz.

Del mismo modo se ingresó una señal con una frecuencia de 2000 Hz, en este caso la variable `xpower2` obtiene un valor cercano a 1 y la variable `xpower1` tiene un valor cercano a 0, por lo tanto, la detección de la frecuencia fue correcta. En la Figura 8 se muestra la detección de la frecuencia de 2000 Hz.

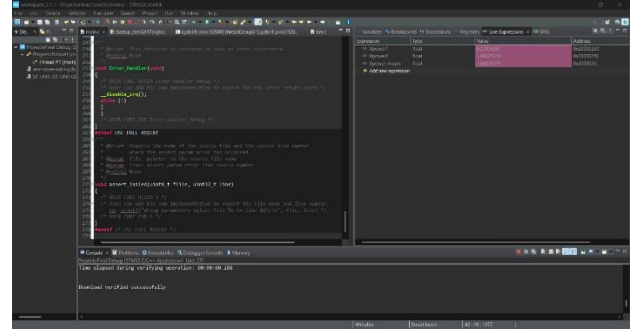


Figura 8. Detección de frecuencia de 2000 Hz.

En un principio se declaró un numero de muestras de 100 por lo tanto, si se ingresaba una frecuencia de 800 Hz, la variable `xpower1` no marca un valor cercano a 1, esto se debe a la relación que se utiliza para calcular el coeficiente k , la cual involucra la frecuencia de muestreo y la frecuencia que se busca encontrar. En la Figura 9 se ilustran los valores de las variables con la frecuencia de 800 Hz.

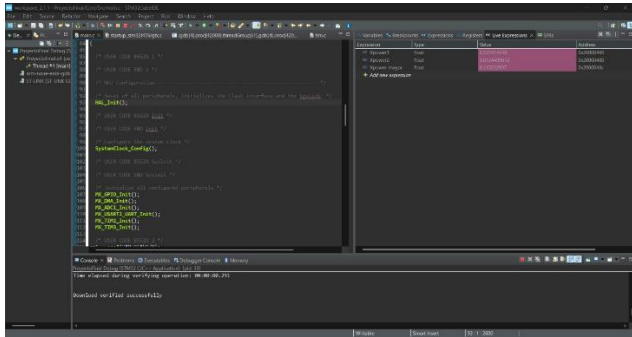


Figura 9. Detección de frecuencia de 800 Hz.

VI. CONCLUSIONES

Al realizar las pruebas se detectó un factor importante, el número de muestras y la frecuencia de muestreo definen la resolución con la cual se pueden detectar las frecuencias de entrada, al tener una frecuencia de 2 KHz, con un numero de muestras de 200, se tendría una resolución de 100, esta fórmula sale de la variable k que se declaró en la sección de introducción. Si se quiere tener una mayor resolución se debe incrementar el número de muestras, así se obtiene una detección de frecuencias más precisa y esto sirve en el caso que se requieran detectar dos frecuencias diferentes de valores muy cercanos. Otro aspecto importante es considerar la señal de entrada, ya que esta no debe de tener magnitudes negativas para no dañar la tarjeta, es por esto que se le agrega un offset y la magnitud de la señal debe de ser menor a 3.3 V.



VII. BIBLIOGRAFÍAS

- [1] J. A. Cortés Osorio, J. A. Mendoza Vargas, y J. A. Muriel Escobar, «Alternativa al análisis en frecuencia de la FFT mediante el algoritmo Goertzel», *Scientia Et Technica*, vol. XVI, núm. 44, pp. 217-222, abr. 2010. [En línea]. Disponible en: <http://www.redalyc.org/articulo.oa?id=84917316040>
- [2] G. C. L. (LU7HZ), «El fantástico filtro Goertzel», *El Blog de LU7HZ*, feb. 2015. [En línea]. Disponible en: <https://lu7hz.blogspot.com/2015/02/el-fantastico-filtro-goertzel.html>. [Consultado: 14-jun-2026].
- [3] Wikipedia, «Goertzel algorithm», *Wikipedia, The Free Encyclopedia*, may. 2026. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Goertzel_algorithm. [Consultado: 14-jun-2026].
- [4] Texas Instruments, «Modified Goertzel Algorithm in DTMF Detection Using the TMS320C80», Note de aplicación SPRA066, Dallas, TX, EE. UU. [En línea]. Disponible en: <http://focus.ti.com/lit/an/spra066/spra066.pdf>. [Consultado: 14-jun-2026].
- [5] A. V. Oppenheim y R. W. Schaffer, *Procesamiento de Señales en Tiempo Discreto*, 3ra ed. Madrid, España: Pearson Educación, 2010.